Shared GKE Tenant Onboarding Guide

Introduction

Welcome to the onboarding guide for the **Shared GKE Platform**. This document is intended to help new application teams (Tenants) onboard their services onto DevEx Multi-Tenant Kubernetes Clusters (Shared GKE Clusters) hosted on GCP's Google Kubernetes Engine(GKE).

This guide provides:

- Understanding of which shared cluster a tenant will be part of.
- An overview on DevEx CI/CD pipeline using GitHub Actions.
- Configuration standards and GitHub repo structure.

By the end of this guide, your team should be able to

- Deploy the GKE with minimal manual intervention.
- Manage and maintain your manifests repos in a structured, scalable manner.
- Leverage DevEx tools and automation.

> If at any point you need help with CI/CD setup (or) patterns, reach out to DevEx platform team #devex-compute-customers, DevSecOps team #pcw-devex-devsecops-engagement-team, DevEx CI/CD team #community-cicd channels on Slack.
>
> DevEx mission statement: devex-cicd-doc.cvshealth.com

Manifest Repo

This manifest repository holds all GKE deployment configurations and namespace related configurations for every tenant onboarding to their respective LOB specific shared cluster. Each cluster will be having a manifest repository, where is holds the Kubernetes manifests for the resources (applications, namespace-config, services etc) deployed on that cluster.

You can find your cluster's manifest repo location **here**.

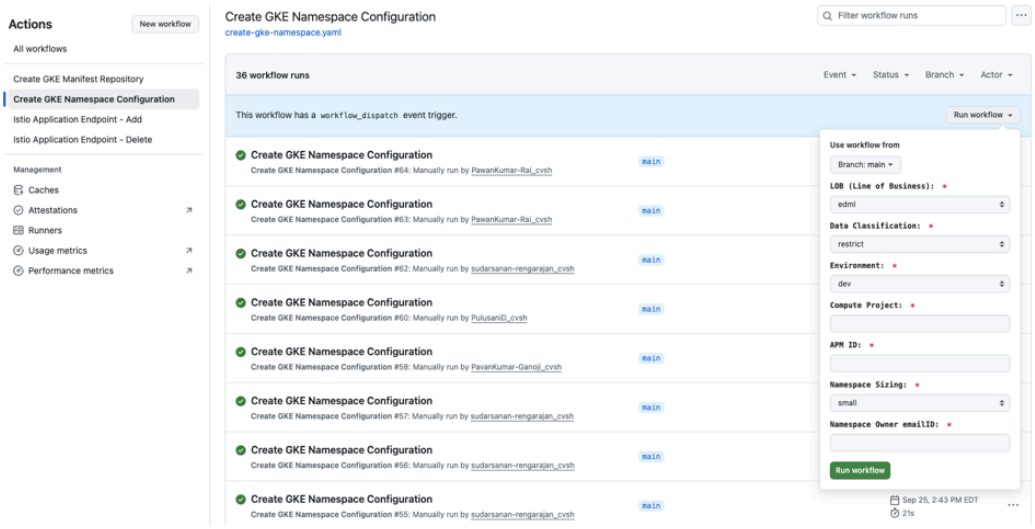
> **Note:** Ensure you have a branch created for your namespace on your cluster specific Manifest repo, If not, you may need to create a separate branch using this automation.

Namespace Configuration

Namespace configuration is nothing but a branch name of the manifest repo. Each branch represents a namespace in that cluster.

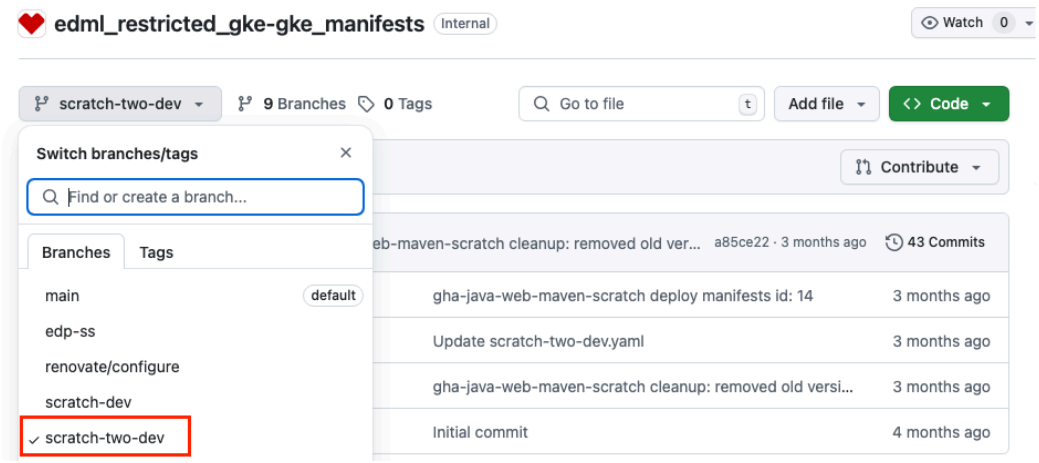
If you don't see a branch name which matches the namespace where you want to deploy your application, then create a branch with the namespace name.

You should use this automation to create a namespace branch in your manifest repo.



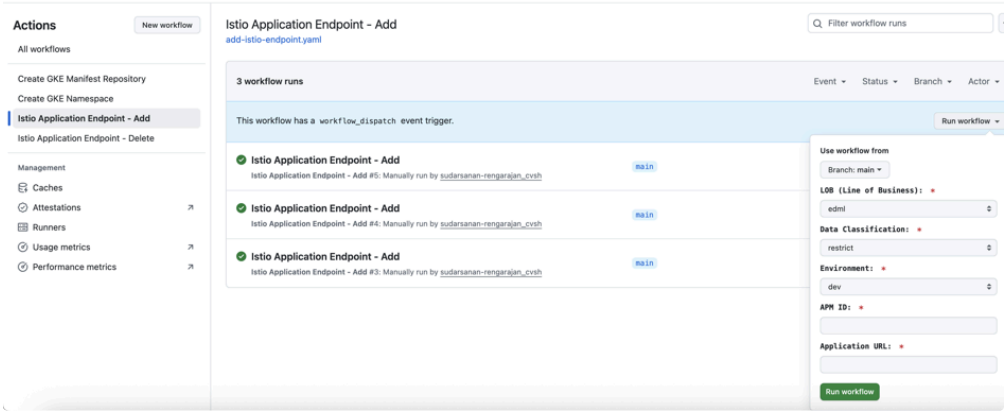
You can refer this self service guide for GKE namespace config in manifest repo (you still need to follow entire document in order to create actual namespace in respective gke cluster).

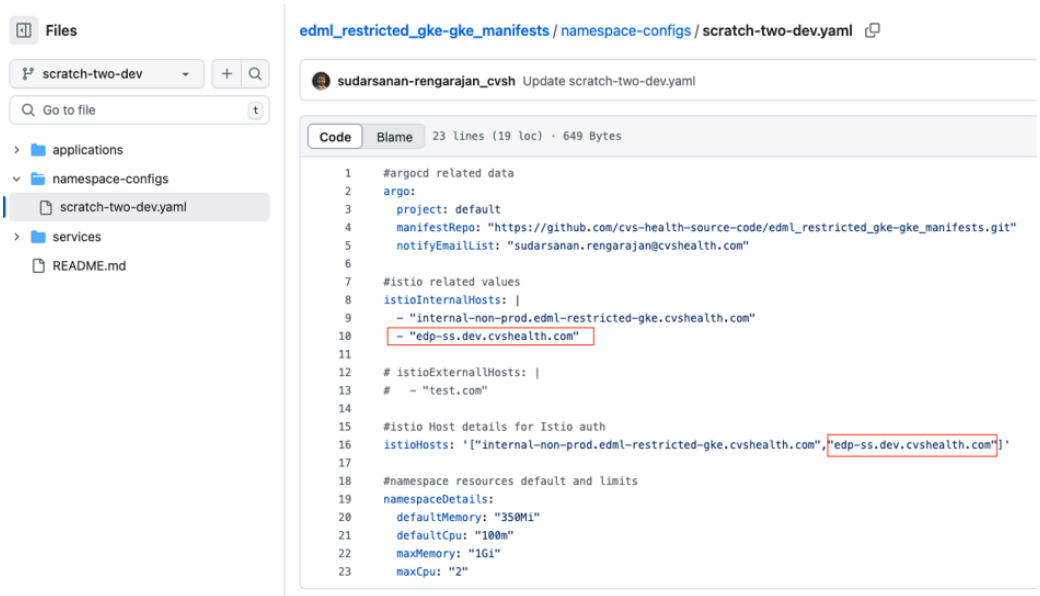
> **Note:**
>
> - It is recommend that an app team should have only one namespace config in manifest repo.



Application Endpoints Configuration

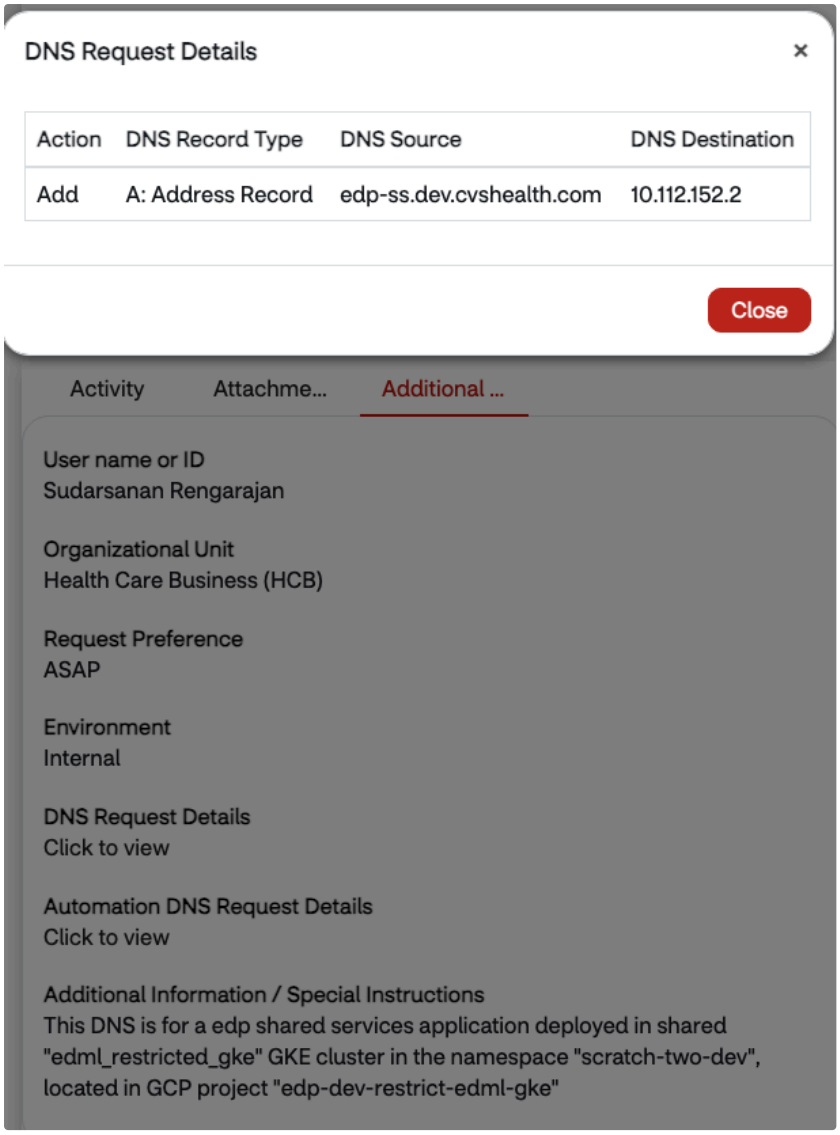
- Make sure application endpoints are added to Istio config provided in namespace-config manifest.
- You can use this automation for adding Istio endpoint to your manifest repo.





Note: Ensure to use cvshealth.com domain for for all edp shared GKE Clusters , DevEx not allowing to add *.aig.aetna.com domains .

- Raise a DNS request for application endpoints. DNS request can be submitted here.
  Note: The IP on the below request is the Ingress IP address of the GKE cluster “edml-restricted-gke”on GCP project “edp-dev-restrict-edml-gke”. You should use the Ingress IP of your respective GKE cluster. Ingress IP can be found here for your respective cluster.

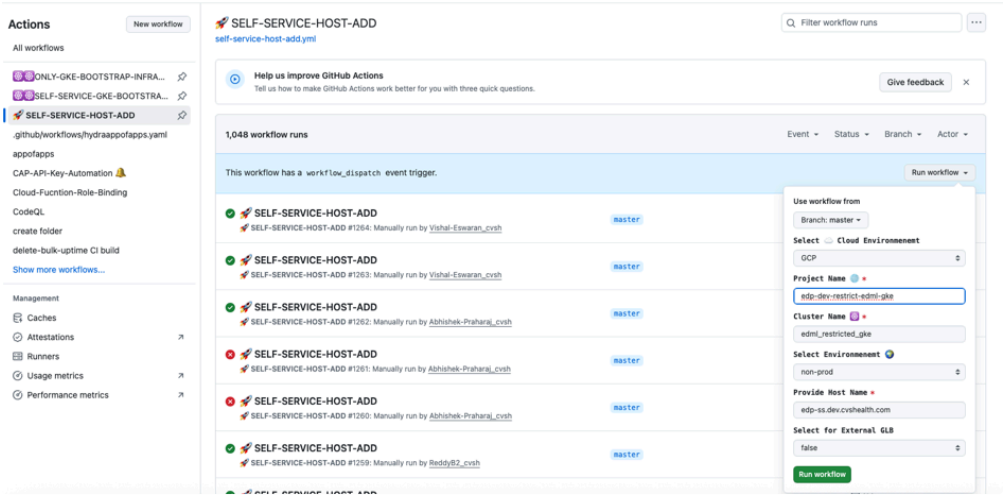


Make sure DNS mapping done correctly before proceeding workflow automation .

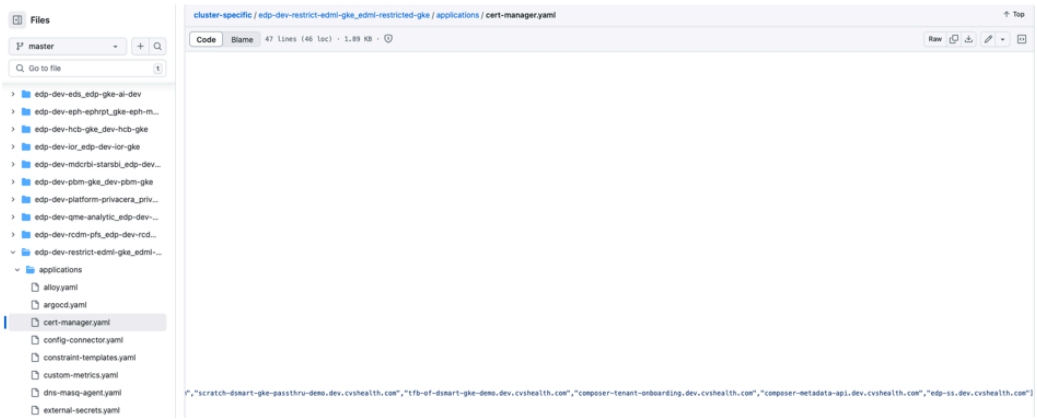
you can verify the DNS mapping by checking https://dnsq.dev.cvshealth.com/

- Run this workflow automation for cert manager configuration for that new endpoint.
  Make sure DNS mapping done correctly before proceeding workflow automation .

  you can verify the DNS mapping by checking https://dnsq.dev.cvshealth.com/



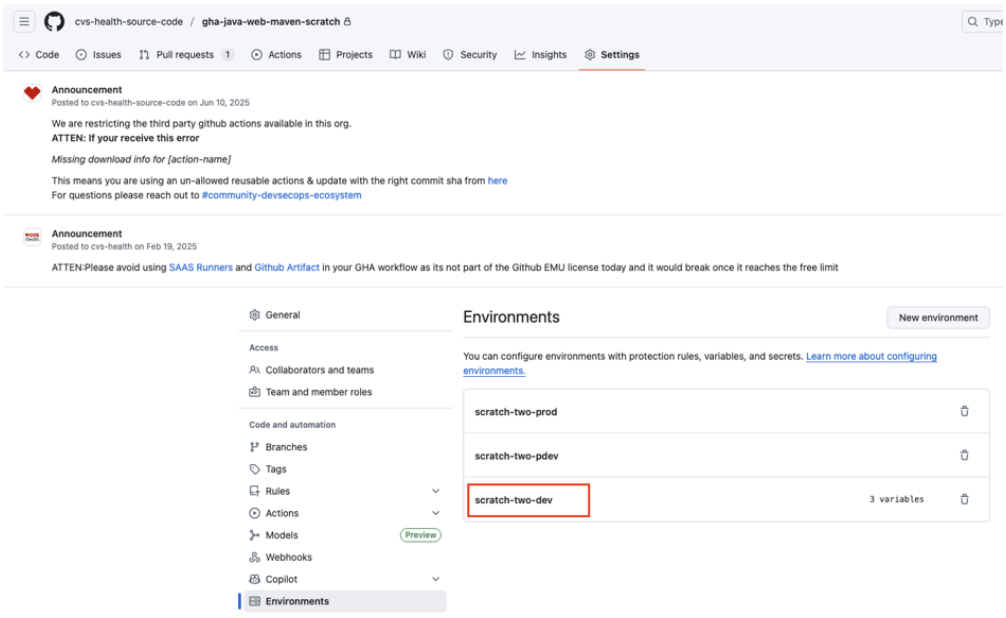
- Ensure that post workflow run, your new domain gets added to the `cert-manager.yaml` in [cluster-specific] repo of your cluster.



One Time Setup for Every Repository

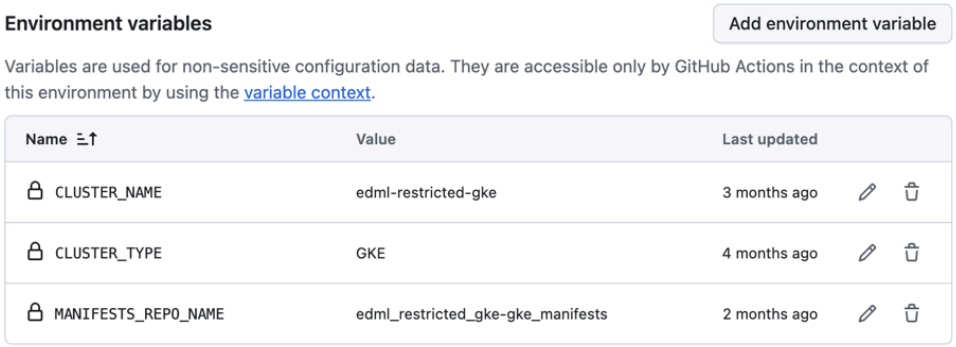
Follow the below steps specified to configure the one time setup for each repository for your team. You need to have **admin** access to your repository to configure the below steps :

1. To create a new environment, go to the repository's **Settings**, select **Environments** from the left menu, click **New environment**, and enter your namespace name (e.g., `scratch-two-dev`).

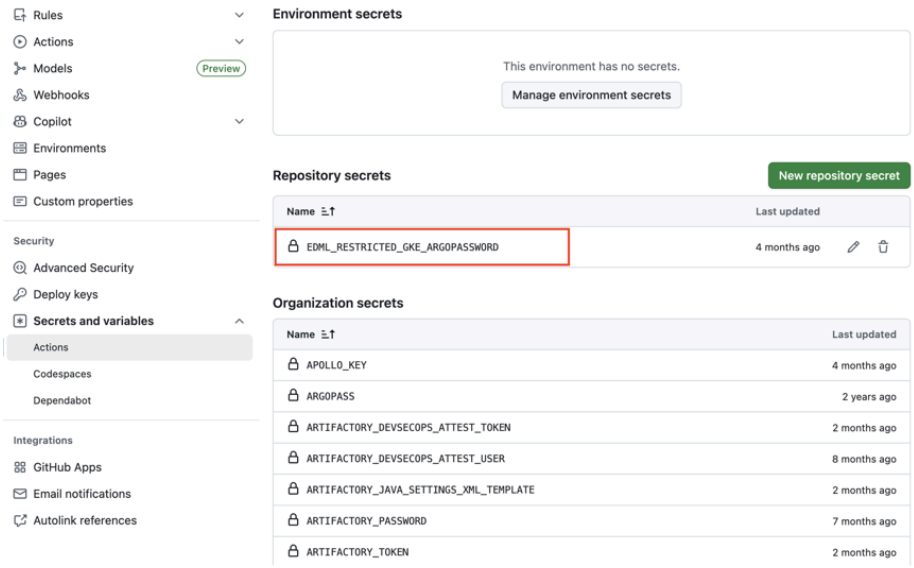


2. To configure environment variables, select the desired environment (e.g., `scratch-two-dev`), click **Add environment variable**, and define the following variables with cluster specific values:
   - `CLUSTER_NAME`
   - `CLUSTER_TYPE`
   - `MANIFESTS_REPO_NAME`

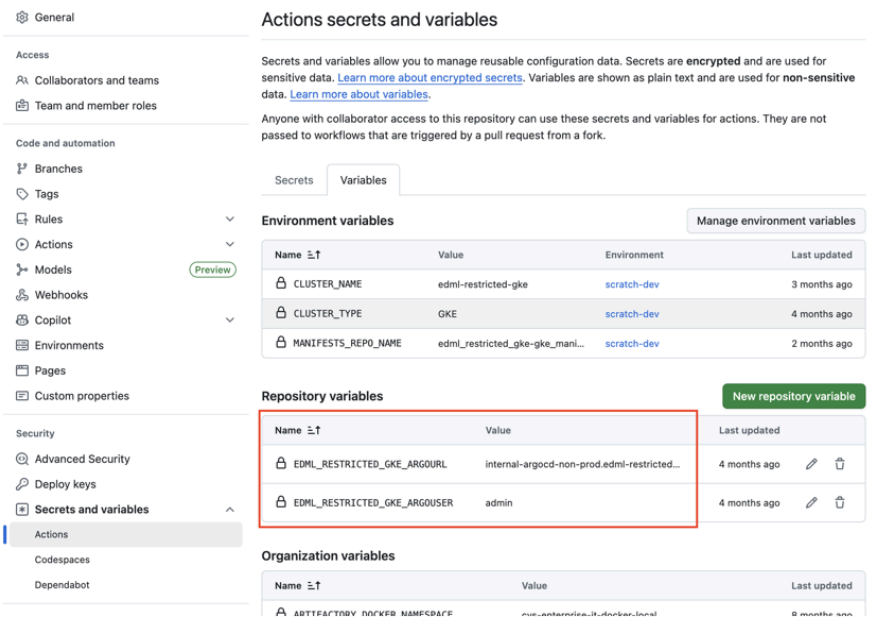
   Ensure the variable name as following the same naming standard and conventions.



3. To configure Repository secret, select "**Secrets and variables**" from the left menu and select the sub-menu "**Actions**", click "**New repository secret**" and add the secret name in the pattern "**<<CLUSTER_NAME>>_ARGOPASSWORD**" based on the cluster name, and configure it with the admin password of ArgoCD.



4. To configure Repository variables, select “**Variables**“ tab next to secrets tab, click “**New repository variable**“ and add the variables name in the pattern “**<<CLUSTER_NAME>>_ARGOURL**“, “**<<CLUSTER_NAME>>_ARGOUSER**“ based on the cluster name, and added it with the related ArgoCD configuration values.



5. Ensure all the custom properties are added as per devex standards https://aetnao365.sharepoint.com/sites/Application-Ecosystem-Security-and-Threat-Management/SitePages/Application-Mapping.aspx Connect your OneDrive account

GitHub Repo Structure & Standards

Source Code Repo

Each application team is responsible for maintaining a dedicated source code repository, which contains the application code base and CI/CD workflows configurations.

Path for holding CI workflows: ***.github/workflows***

CI

Note: make sure to always use "@latest" for all the deployments (node/maven/python) in CI.yaml . this is to ensure you are using always devex latest workflows.

Recommended Folder Layout for CI workflow

```
name: Run Docker Pipeline
on: [workflow_dispatch]
jobs:
  docker_build:
    uses: cvs-health-source-
code/gha_workflow_actions/.github/workflows/maven_docker.yaml@latest
    with:
      JAVA_VERSION: "20"
      JAVA_BUILD_COMMAND: "mvn clean package"
      SAST_TOOL: snyk
      SCA_TOOL: snyk
    secrets: inherit
```

DevEx Documentation: CI Workflow Step-By-Step Process

Sample Projects

Java Tech Stack Example

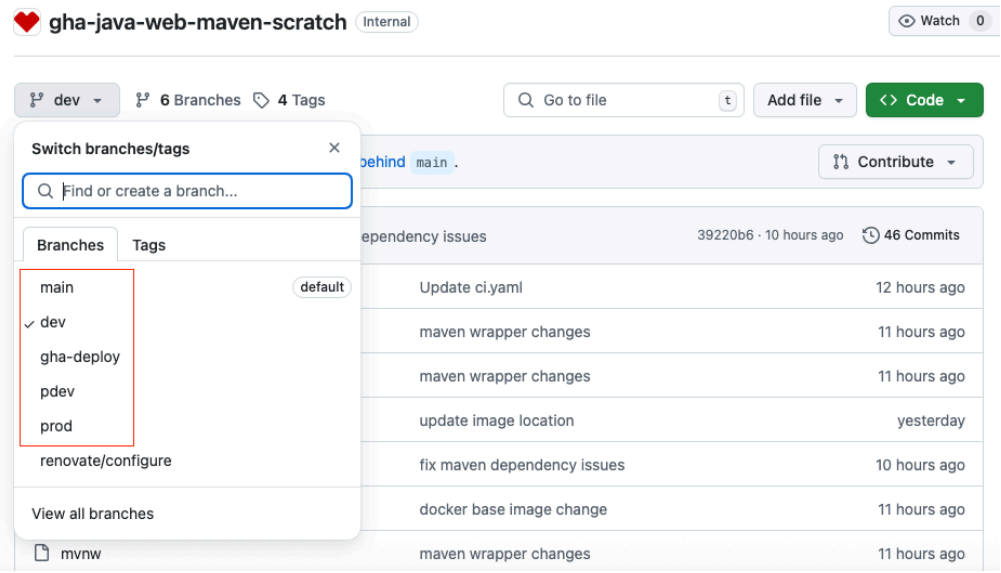
https://github.com/cvs-health-source-code/gha-java-web-maven-scratch Connect your Github account

Python Tech Stack Example

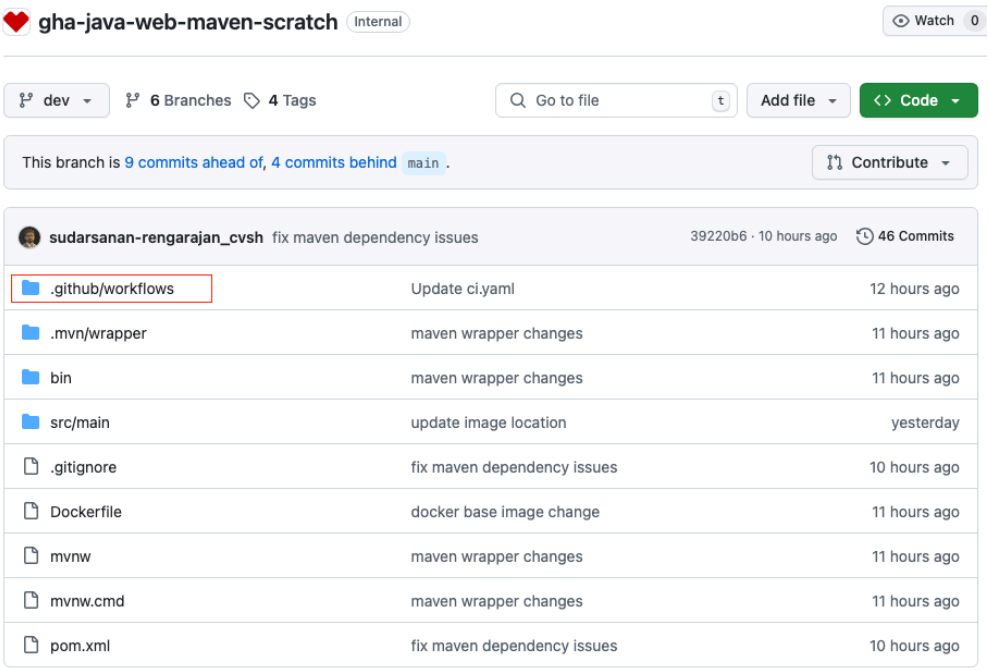
https://github.com/cvs-health-source-code/scratch-340b-dsmart-demo Connect your Github account

https://github.com/cvs-health-source-code/rxomni-drug-search Connect your Github account

1. Create branches for your environment dev/qa/pdev/pqa/prod (Will explain about gha-deploy branch during the CD process)

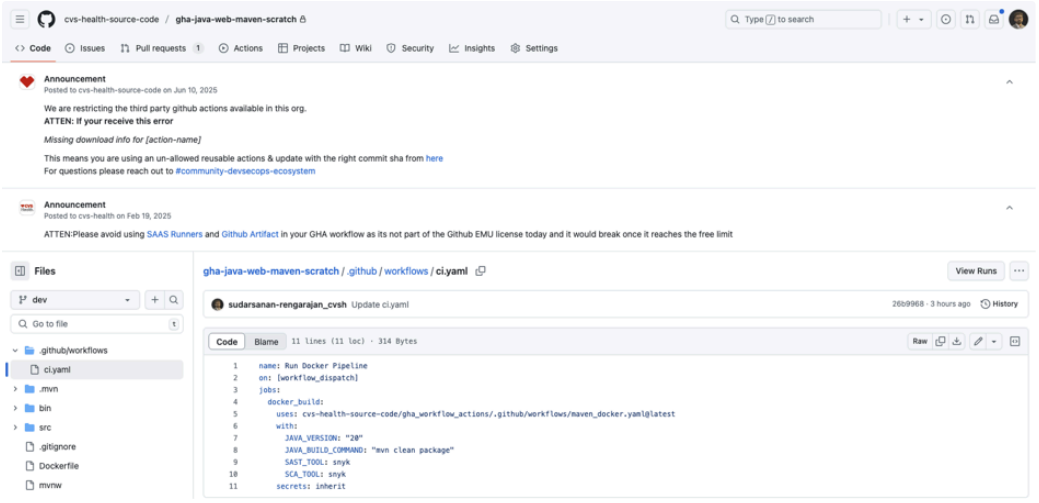


2. Create ***.github/workflows*** in root directory of the respective env branch.



3. Create a ci.yaml file in ***.github/workflows*** directory.

   Refer this reusable workflows based on the tech stack you use for your application, in this example we have used maven_docker workflow yaml.

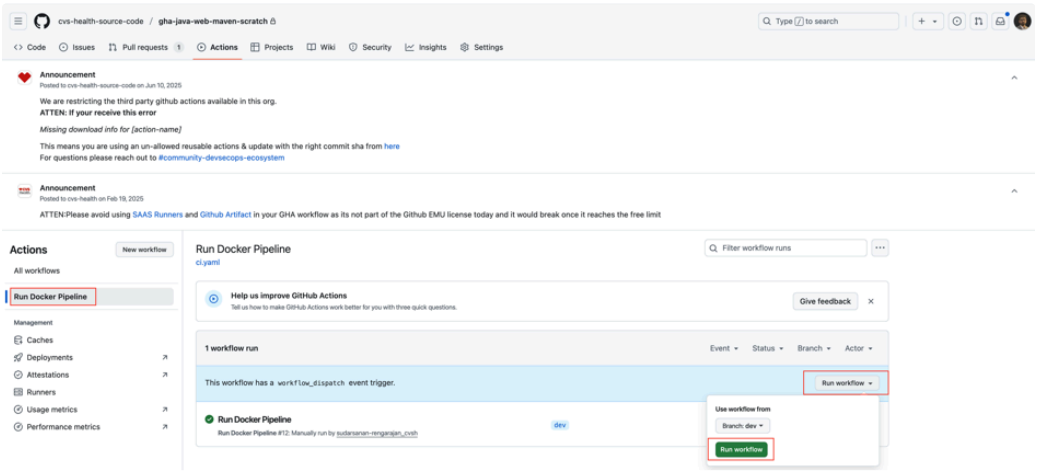


Jfrog

Document for JFrog access: DevEx Jfrog Artifactory Access and Life Cycle Policy | Jfrog Access groups

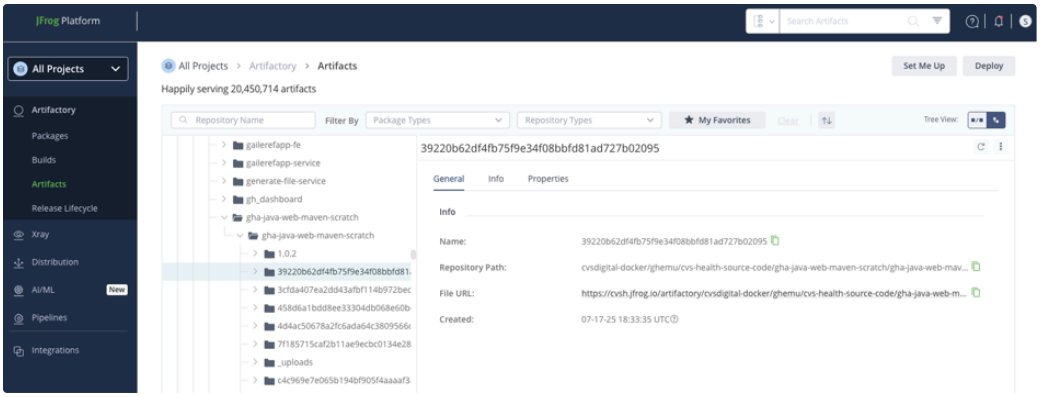
Steps to push the image to JFrog:

- Navigate to “Actions” tab and click “Run Docker Pipeline” workflow and select the “Run workflow“ dropdown to trigger the CI process by clicking on “Run workflow” button.

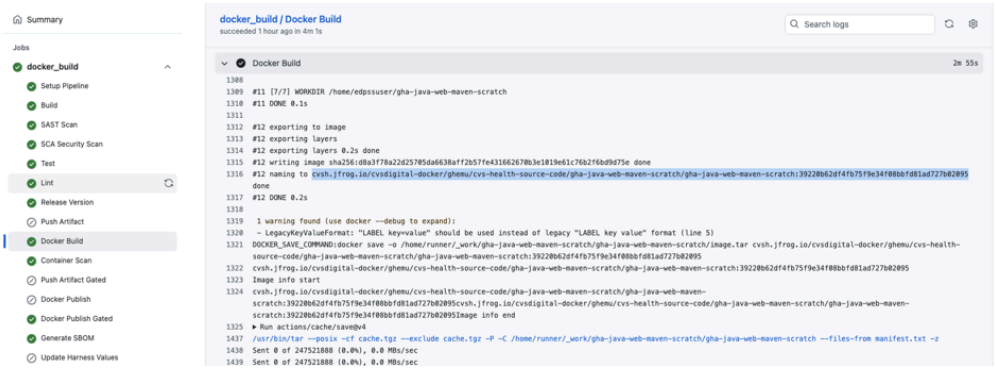


- This process will go through all the stages in CI to push your application image to jfrog registry.

  In this example, it will push it to this location `cvsh.jfrog.io/cvsdigital-docker/ghemu/cvs-health-source-code/gha-java-web-maven-scratch`



The version of the image can be found from the gha workflow logs under docker build section as well.



Refer Snyk CI setup for GHA

https://aetnao365.sharepoint.com/sites/Application-Ecosystem-Security-and-Threat-Management/SitePages/CI-Setup.aspx Connect your OneDrive account

CD

Recommended Folder Layout for CD workflow

```yaml
name: App Deployment

on:
  workflow_dispatch:
  push:
    branches: [gha-deploy]
    paths: ['**/values/v1.yaml']

permissions:
  id-token: write
  contents: write
  pull-requests: write

jobs:
  detect-changes:
    runs-on:
      group: cvs-linux-self-hosted
    outputs:
      CHANGED_ENVIRONMENTS: ${{ steps.set-matrix.outputs.CHANGED_ENVIRONMENTS }}
    steps:
```

```yaml
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Detect changes
        id: changes
        uses: dorny/paths-filter@de90cc6fb38fc0963ad72b210f1f284cd68cea36
        with:
          base: ${{ github.ref_name }}
          filters: |
            scratch-two-dev_v1: '**/deploy-configs/scratch-two-dev/values/v1.yaml'
            scratch-two-pdev_v1: '**/deploy-configs/scratch-two-pdev/values/v1.yaml'
            scratch-two-prod_v1: '**/deploy-configs/scratch-two-prod/values/v1.yaml'

      - name: Build Environment Matrix
        id: set-matrix
        run: |
          ENVS=()

          if [ "${{ steps.changes.outputs.scratch-two-dev_v1 }}" = "true" ]; then
            ENVS+=("{\"ENV_NAME\":\"scratch-two-dev\",\"HELM_FILE_VERSION\":\"v1\"}")
          fi

          if [ "${{ steps.changes.outputs.scratch-two-pdev_v1 }}" = "true" ]; then
            ENVS+=("{\"ENV_NAME\":\"scratch-two-pdev\",\"HELM_FILE_VERSION\":\"v1\"}")
          fi

          if [ "${{ steps.changes.outputs.scratch-two-prod_v1 }}" = "true" ]; then
            ENVS+=("{\"ENV_NAME\":\"scratch-two-prod\",\"HELM_FILE_VERSION\":\"v1\"}")
          fi

          if [ ${#ENVS[@]} -gt 0 ]; then
            JSON=$(IFS=,; echo "[${ENVS[*]}]")
            echo "CHANGED_ENVIRONMENTS=$JSON" >> $GITHUB_OUTPUT
          else
            echo "CHANGED_ENVIRONMENTS=[]" >> $GITHUB_OUTPUT
          fi

  deploy:
    uses: cvs-health-source-code/gitops-cd-workflow/.github/workflows/helm-deploy.yaml@v1
    needs: detect-changes
    if: needs.detect-changes.outputs.CHANGED_ENVIRONMENTS != '[]'
    strategy:
      fail-fast: false
      matrix:
        environment: ${{ fromJson(needs.detect-changes.outputs.CHANGED_ENVIRONMENTS) }}
    secrets: inherit
    with:
      app-name: "gha-java-web-maven-scratch"
      environment: ${{ matrix.environment.ENV_NAME }}
      helm-template-repo: ${{ vars.HELM_TEMPLATE_REPO }}
      helm-file-version: ${{ matrix.environment.HELM_FILE_VERSION }}
      helm-template-branch: main
      canary-weight: "0"
      lob: "account"
      change-description: "EDP SS demo application change for production"
      rfc-short-desc: "This is to deploy EDP SS demo application to production"
      change-domain: "Enterprise"
```

```
79        change-impacted-website: "CVS.COM"
```

DevEx Documentation: CD Workflow

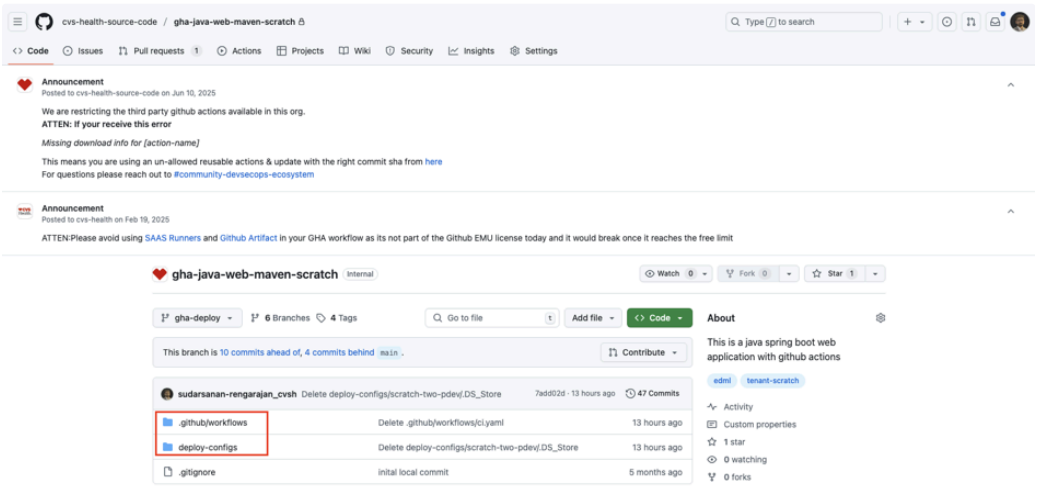
Sample Projects

https://github.com/cvs-health-source-code/gha-java-web-maven-scratch Connect your Github account

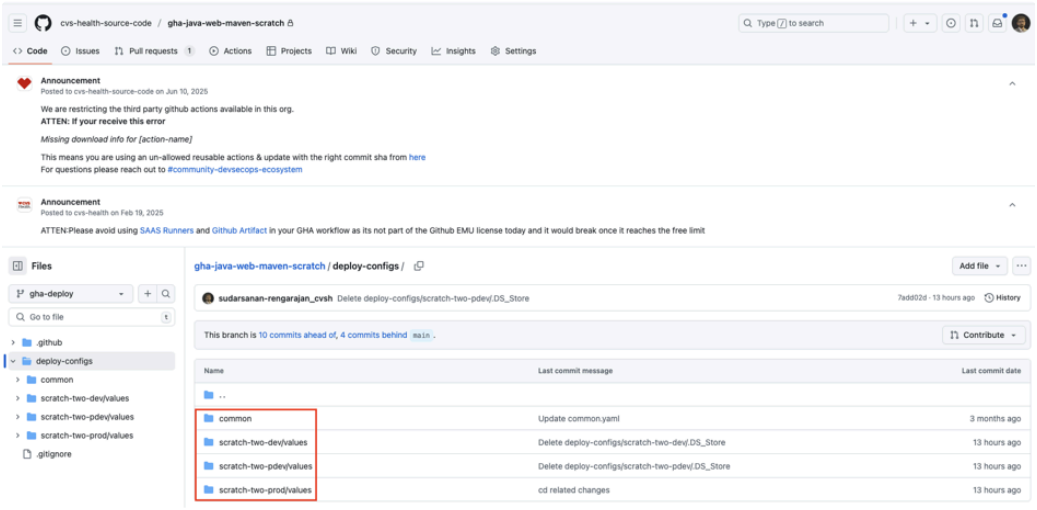
https://github.com/cvs-health-source-code/edp-ss-python-flask-demo/tree/main Connect your Github account

Configuration

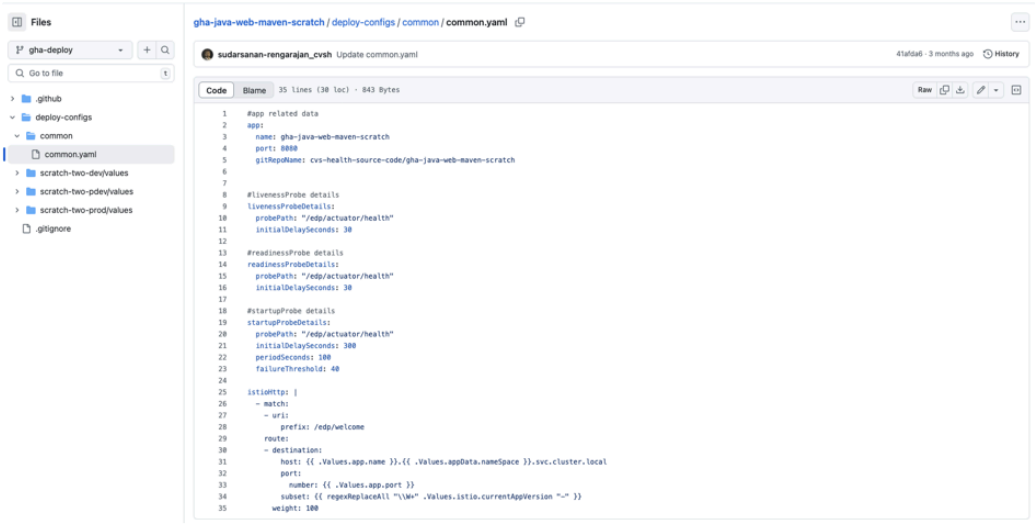
To hold the deployment configuration for Tenants, create a branch `gha-deploy` and create these two folders `./github/workflows` & `deploy-configs`.



`deploy-configs` directory generally has child directories `common` and `<namespace_{n}>/values`.



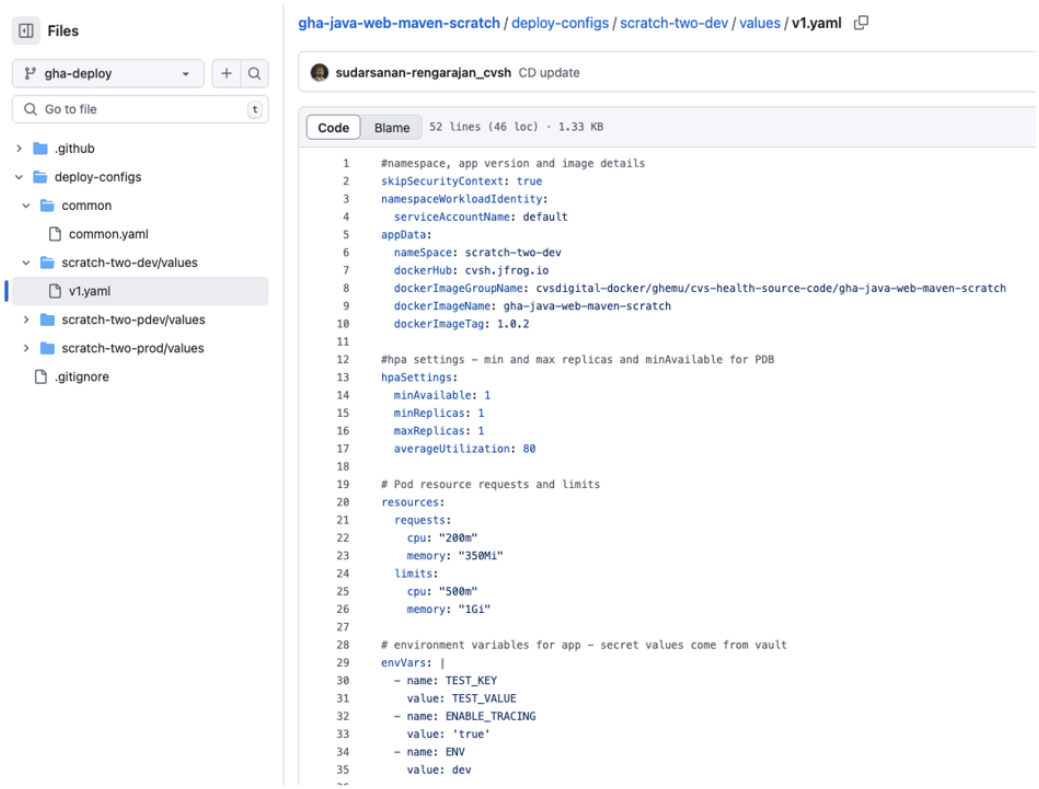
`common` : This directory holds the configuration spec values which are common across all the app deployments.



`namespace` : This directory will hold the application specific configuration values (ex: v1, v2 etc.).

Here in below example `scratch-two-dev` is the namespace in this reference screenshot.

To initiate the deployment of your application, you need to update the `dockerImageTag` at `line 10` as in the below screenshot, with the latest Image tag, that got generated by your last CI workflow run.



Naming Standards

To ensure consistency, observability, and ease of automation, all GKE resource names across tenants should follow these naming patterns:

Resource Type	Naming Pattern	Example
Namespace	<lob>-<appAcronym>-<envname>	edml-edpss-dev edml-edpss-qa edml-edpss-pdev edml-edpss-pqa edml-edpss-prod

Note:

- Please be informed that you appcronym which is retrived from service now will be trimmed to 30 char.

ArgoCD

Every cluster will have a ArgoCD endpoint, which is of pattern `https://internal-argocd-prod.$CLUSTER_NAME.cvshealth.com`. where you can track the

applications deployed on the GKE cluster.
More documentation on ArgoCD: ArgoCD App of Apps

References

Recording can be found here.